

 EMERGENCY RESPONSE

 EVACUATION TRACKING

EVA-NET

Emergency Vector-Aware Network for Evacuation Tracking

Spatial Graph Optimization & P2P Data Fusion

Team Members

Siddhant Agarwal - 25WU0102262

Sampriti Pal - 25WU0101121

Shaurya Dhingra - 25WU0102254

Sayan Bhowmick - 25WU0103035

Soubhik Ghosh - 25WU0101137

Mentor

Kiran Mayee Adavala

Problem Identification I: Infrastructure Collapse

Cellular Grid Destruction



Tower Destruction: Wildfires, extreme winds, and storm surges physically knock out cell towers and high-voltage power grids.



Network Congestion: Millions attempting calls simultaneously cause total cellular packet loss and severe voice/data throttling.



Single Point of Failure: Centralized cloud architectures become entirely unreachable during major disasters.

Navigation Blackouts



Centralized API Blindness: Popular apps (Google Maps, Waze) require persistent cloud servers to query live traffic and road updates.



Deadly Rerouting Suggestions: Without server pings, standard GPS apps route fleeing evacuees directly into active fire fronts or submerged roads.



First Responder Delays: Emergency crews lack real-time local road block visibility.

Problem Identification II: Stale Data

THE FLAW IN STATIC OFFLINE MAPS

While offline maps store base road geometry on phone storage, they are completely blind to real-time dynamic hazards.

CRITICAL OPERATIONAL BOTTLENECKS:

- Unreported Road Blockages: Flash floods wash out rural bridges; fallen trees and power lines sever arterial highways unexpectedly.
- Crowdsourcing Isolation: Evacuees who encounter a hazard cannot upload reports to the cloud without internet connectivity.
- Asymmetric Hazard Knowledge: Drivers 2 miles behind a blockage remain completely unaware until they hit the obstacle themselves.



Real-World Example: The 2023 Maui Wildfire Crisis

THE PROBLEM IN ACTION

On August 8, 2023, the Maui wildfires devastated Lahaina, Hawaii. Thousands of residents attempted evacuation but faced critical navigation failures:

- Cell towers destroyed: 90% of cellular infrastructure in the fire zone went offline within minutes
- GPS apps failed: Google Maps and Waze directed evacuees toward active fire fronts due to stale road data
- No real-time updates: Residents 2 miles behind the fire front had no way to know which roads were blocked
- Emergency response delayed: First responders couldn't coordinate because they lacked real-time road closure visibility

RESULT: Over 100 deaths, many due to evacuation route failures

EVA-NET SOLUTION: With peer-to-peer hazard sharing, evacuees near the fire front would have immediately shared road blockage data with those behind them—enabling life-saving reroutes without any central infrastructure.



Core Objectives & Disaster Scenarios

Project Objectives

Zero Cloud Reliance

Maintain continuous routing capability when internet backbones fail entirely.

Opportunistic Graph Fusion

Merge spatial vector updates peer-to-peer via local ad-hoc wireless links (BLE / Wi-Fi Direct).

Dynamic Safety Weighting

Continuously reweight spatial road graphs to guide evacuees along the safest routes.

Target Disaster Scenarios

Wildfires

Rapidly shifting fire fronts engulfing highways and rural access corridors.

Flash Floods & Storm Surges

Submerged bridges, road washouts, and severe debris accumulation.

Earthquakes & Landslides

Structural building collapses and road ruptures severing main routes.

Methodology: Steps 1-2

Lightweight spatial graph modeling and local sensing work together to keep hazard updates small, fast, and resilient.

Step 1: Lightweight Spatial Graph Representation

Graph Modeling: Road network represented as spatial graph $G = (V, E)$, where V = intersections/junctions and E = directed road segments.

Vector Storage: Lightweight geospatial vector geometries using GeoPandas and Shapely instead of heavy map tiles.

Step 2: Local Sensing & Delta Update Generation

When an evacuee encounters an obstacle such as a flooded street, fallen trees, or active fire:

- User logs blockage or the app infers it via sudden speed drops
- App modifies local edge state and generates a Vector Diff packet (< 5 KB)
- **Vector Diff** = {Edge_ID, Hazard_Type, Timestamp, Confidence_Score}

Methodology: Steps 3-5

Step 3: Opportunistic P2P Data Fusion

- Ad-Hoc Wireless Transport: Bluetooth Low Energy / Wi-Fi Direct (infrastructure-free)
- Transit Propagations: When devices come within range (~10-100 meters), automated background handshake triggers
- Vector Swap: Devices exchange missing Vector Diffs, spreading hazard knowledge transitively

Step 4: Decentralized Graph Reweighting

Upon receiving new hazard layers, phone updates edge weights using:

$$W_{edge} = W_{base} + H(hazard) + T(staleness)$$

- **W_base**: Baseline travel time/distance
- **H(hazard)**: Penalty multiplier (∞ for complete blockages)
- **T(staleness)**: Time-decay penalty for old unconfirmed reports

Step 5: On-Device Path Calculation & Rerouting

- Runs A* or Dijkstra locally on updated edge-weight matrix
- Rerouting completes in <50ms
- Instantly updates visual path on Folium/Leaflet.js UI without central server

System Architecture Overview

The system is organized into two tightly coupled layers: a local device stack that stores, logs, and prepares routing data, and a P2P mesh network that discovers peers, exchanges hazard updates, and renders live routes.

Layer 1: Local Device Stack

Smartphone Storage

Pre-loaded vector road geometries from OpenStreetMap data.

Graph Engine

NetworkX-based adjacency matrix with dynamic edge weights.

Hazard Logger

Captures user reports and sensor GPS coordinates.

Vector Diff Generator

Creates compact <5KB payloads for transmission.

Layer 2: P2P Mesh Network

Wireless Protocols

BLE (10-100m range) and Wi-Fi Direct (100-300m range).

Peer Discovery

Automatic device detection and handshake protocol.

Conflict Resolution

Time-decayed confidence scoring for hazard merging.

UI Renderer

Folium/Leaflet.js for real-time route visualization.

- ❑ Local devices generate and update routing data, while the mesh network propagates verified hazard information and keeps the route view synchronized.

Literature Review: EVA-NET

This review synthesizes four research areas relevant to EVA-NET: centralized navigation systems, offline mapping, mobile ad-hoc networking, and dynamic graph reweighting.

CENTRALIZED CLOUD NAVIGATION

Standard systems. Google Maps and Waze rely on centralized architecture.

Current state. Cloud servers process GPS data and globally reweight road segments (*Wang et al., 2021*).

Failure. Infrastructure collapse and packet congestion leave evacuees with static routes (*Chen et al., 2020*).

STATIC OFFLINE MAPPING

Standard maps. Pre-downloaded vector tiles store baseline spatial graph $G=(V,E)$ locally.

Current state. Offline Dijkstra and A* work without servers.

Failure. Static datasets cannot incorporate real-time hazards (*Li et al., 2022*) and may route users into danger zones.

MOBILE AD-HOC NETWORKS (MANETs)

Infrastructure-free propagation. BLE and Wi-Fi Direct enable opportunistic exchange (*Conti & Giordano, 2014*).

Current state. Epidemic Routing and DTN pass raw text messages (*Saha et al., 2018*).

Gap. No Geospatial Vector Fusion; full map images over BLE create congestion.

DYNAMIC GRAPH REWEIGHTING

Dynamic routing. Time-varying edge weights are addressed by D* Lite (*Koenig & Likhachev, 2002*).

Current state. Incremental shortest-path updates respond as obstacles are discovered.

Gap. Little work supports asynchronous vector delta merging across decentralized moving nodes without conflicts.

- ❑ EVA-NET fills the key gap: a decentralized, low-bandwidth, hazard-aware spatial graph that updates reliably during infrastructure failure.

Detailed Comparison & EVA-NET's Technical Advantages

EVA-NET addresses critical shortcomings in existing navigation and emergency communication systems, offering robust solutions for disaster scenarios.

Existing Limitations

Centralized Cloud Navigation

Services like Google Maps and Waze rely on central cloud servers and functional telecommunication infrastructure, failing entirely during power outages and physical destruction of cellular networks.

Static Offline Maps

Pre-downloaded maps lack dynamic hazard awareness. They direct evacuees into danger zones if highways are flooded or blocked by fallen debris, as they cannot incorporate real-time updates.

Standard Emergency Ad-Hoc Networks

Current ad-hoc networks prioritize peer-to-peer messaging (e.g., SOS texts). They lack an integrated spatial GIS engine to translate incoming reports into optimized navigation routes.

EVA-NET's Advantages

Zero-Cloud Dependency

EVA-NET operates purely on local device computation and peer-to-peer radio protocols (Bluetooth LE / Wi-Fi Direct), continuing to function and share real-time road conditions even when cell towers are down.

Dynamic Hazard Adaptability

EVA-NET converts static vector maps into live, self-updating graphs. By swapping lightweight "Vector Diffs" with passing evacuees, every device stays updated on real-time hazards without needing internet access.

Integrated Spatial Graph Fusion

Instead of disjointed alerts, EVA-NET automatically ingests P2P updates into a spatial vector graph, reweights edge travel costs, and executes instant path recalculations ($< 50\text{ ms}$) on device.

Proposed Solution & Novelty

PEER-TO-PEER SPATIAL VECTOR FUSION

Zero-cloud edge GIS. Smartphones run the full spatial engine locally.

Local mesh. Vector geometries are updated, merged, and processed via BLE/Wi-Fi Direct.

COMPACT BINARY VECTOR DIFFS

<5 KB payloads. Transfer ultra-compact vector diffs instead of full maps.

Payload. {Edge_ID, Hazard_Type, Timestamp, Confidence_Score}

DECENTRALIZED GRAPH REWEIGHTING

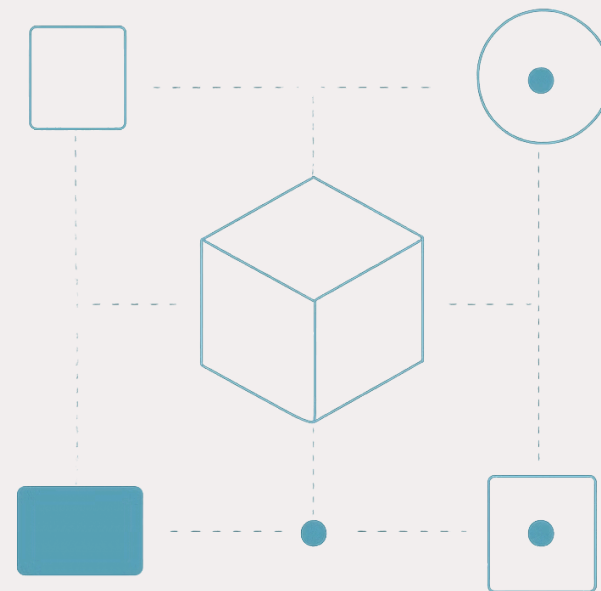
Temporal decay. $W_{\text{edge}} = W_{\text{base}} + H(\text{hazard}) + T(\text{staleness})$

Adaptive routing. Old or unconfirmed reports are automatically down-weighted.

ASYNCHRONOUS EDGE MATRIX CONSENSUS

Pairwise reconciliation. Devices resolve conflicts locally with timestamps and confidence scores.

No master node. Lightweight sync works without a centralized database.



Tech Stack & Antigravity Simulation

CORE STACK

Python (GeoPandas, Shapely, NetworkX), FastAPI, WebSockets, and Folium/Leaflet.js.

SYNTHETIC CITY GRIDS

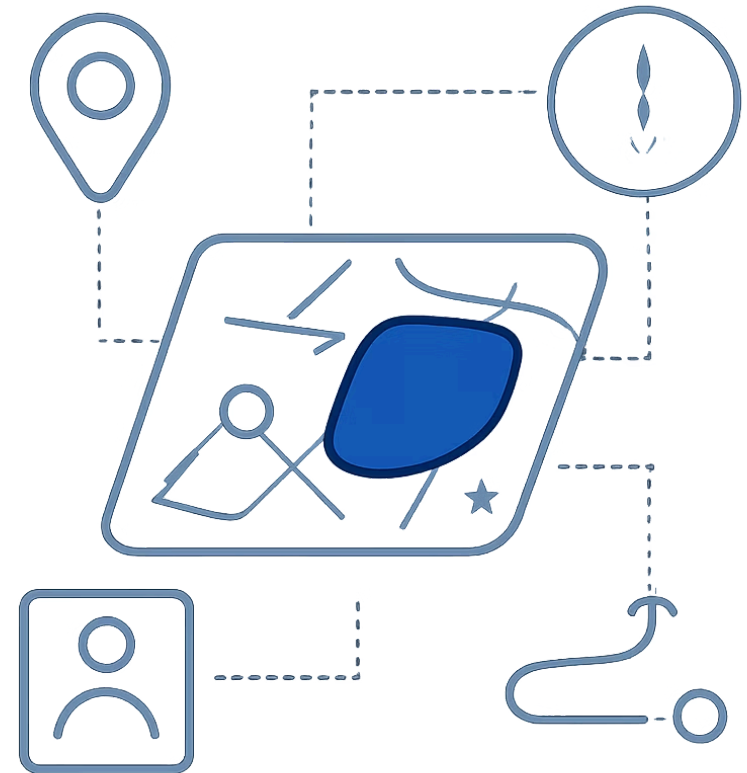
Generates realistic urban road graphs populated with simulated evacuee agents.

DYNAMIC HAZARD SPAWNING

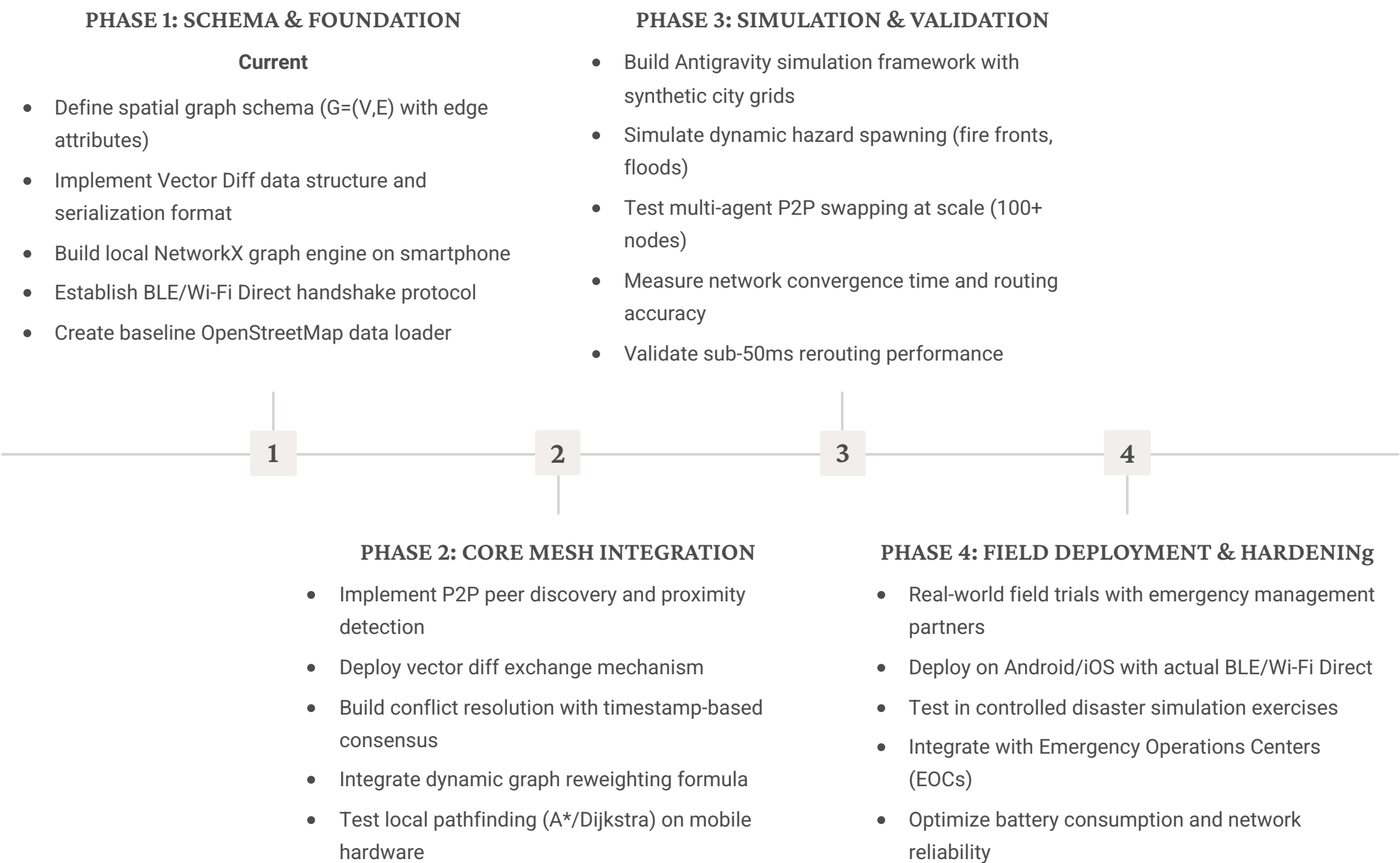
Simulates expanding fire fronts and flash floods blocking edges over time.

P2P SWAPPING SIMULATOR

Simulates proximity encounters where moving nodes exchange vector diffs.



EVA-NET Development Roadmap



Thank You for Your Attention

Questions & Discussion

Resilience

Building decentralized systems that save lives

Innovation

Spatial graph optimization meets peer-to-peer networks

Impact

Enabling life-saving evacuations without infrastructure

Future

Decentralized navigation for every disaster scenario